CPSC 250
Big Program One: Constructing a 32-Bit Floating Point Representation

Edward Brash

1 Purpose of the Program

In this set of notes, we will build a larger Python program whose purpose is to illustrate how a real number can be represented in binary floating point form.

The goal is not to replace Python's built-in floating point system. Python already does this for us. Instead, the goal is pedagogical:

> We want to understand, step by step, how a decimal floating point number can be decomposed into a binary representation involving a sign bit, exponent bits, and mantissa bits.

This program brings together many of the ideas reviewed earlier in the course.

2 Skills Needed

To build this program, we will need several pieces of Python knowledge.

1. Knowledge of primitive data types: integers, floats, and strings.
2. Knowledge of functions: functions we write ourselves, built-in functions, and class methods.
3. Knowledge of how floating point numbers are stored.
4. Loops: both `for` loops and `while` loops.
5. Branching: `if`, `else`, and logical conditions.
6. Getting input from the user.
7. Producing formatted output using `print()`.

3 Overall Program Design

The program will ask the user for a decimal floating point number, then construct an approximate binary floating point representation.

The high-level design is:

1. Get a floating point number from the user.
2. Ask how many binary places should be used after the binary point.

3. Write a function that converts the integer part of the number to binary.
4. Write a function that converts the fractional part of the number to binary.
5. Determine the mantissa and exponent.
6. Construct a 32-bit floating point representation.

4 Step 1: Get a Floating Point Number

We begin by asking the user for a decimal floating point value.

```
n = input("Enter your floating point value: ")
```

Important:

> The value returned by `input()` is always a string.

Therefore, if the user enters `3.125`, then initially Python stores the text string `"3.125"`, not the numeric value `3.125`. To treat this value numerically, we convert it:

```
n = float(input("Enter your floating point value: "))
```

5 Step 2: Ask for the Number of Binary Decimal Places

Next, we ask how many places should be included after the binary point.

For example, the decimal number

$$0.1_{10}$$

has a binary expansion that does not terminate cleanly:

$$0.1_{10} = 0.0001100110011\ldots_2$$

Therefore, in a practical program, we must decide how many binary digits after the point we want to keep.

```
p = int(input("Enter the number of binary places in the result: "))
```

The variable `p` is an integer. For example, if `p = 6`, then we will compute six binary digits after the binary point.

6 Step 3: Write a Conversion Function

We want to write a function that takes the number `n` and the number of binary places `p`, then returns the binary representation as a string.

```
def get_binary_rep(n, p):
    ...
    return result
```

The output will be a string because binary representations such as `11.001000` are easiest to construct as text.

7 Working Example

We will use:

$$3.125_{10}$$

and compute the binary representation to six places after the binary point.

$$3.125_{10} = 11.001000_2$$

The rest of these notes explain how to produce this result algorithmically.

8 Breaking the Problem into Pieces

To convert a decimal number to binary, we separate it into two parts:

$$3.125 = 3 + 0.125$$

Then we handle each part separately:

1. Convert the integer part to binary.
2. Convert the fractional part to binary.
3. Join the two pieces with a binary point.

9 Converting the Integer Part to Binary

For the integer part:

$$3_{10} = 11_2$$

More generally, we convert an integer to binary using repeated integer division by 2.

9.1 Example: Converting 13 to Binary

Consider:

$$13_{10}$$

We repeatedly divide by 2 and track the remainder.

Number	Integer Division by 2	Remainder
13	$13 \,//\, 2 = 6$	$13 \,\%\, 2 = 1$
6	$6 \,//\, 2 = 3$	$6 \,\%\, 2 = 0$
3	$3 \,//\, 2 = 1$	$3 \,\%\, 2 = 1$
1	$1 \,//\, 2 = 0$	$1 \,\%\, 2 = 1$

The remainders are produced from right to left:

$$13_{10} = 1101_2$$

9.2 Algorithm

The algorithm is:

1. Start with an integer `num`.
2. Set the result string to the empty string.
3. While `num > 0`:
   (a) Compute `digit = num % 2`.
   (b) Add this digit to the front of the result string.
   (c) Replace `num` with `num // 2`.

9.3 Python Function

```
def convert_integer_to_binary(num):
    # Convert a nonnegative integer to a binary string.

    if num == 0:
        return "0"

    result = ""

    while num > 0:
        digit = num % 2
        result = str(digit) + result
        num = num // 2

    return result
```

The line

```
result = str(digit) + result
```

adds the new digit to the front of the string because the remainders appear in reverse order.

10 Converting the Fractional Part to Binary

Now consider the fractional part:

$$0.125_{10}$$

Since

$$0.125 = \frac{1}{8},$$

we know:

$$0.125_{10} = 0.001_2$$

To six places:

$$0.125_{10} = 0.001000_2$$

11 Understanding Binary Fractions

Binary fractional places represent powers of two:

$$0.b_1b_2b_3b_4\ldots_2$$

means:

$$b_1\left(\frac{1}{2}\right)+b_2\left(\frac{1}{4}\right)+b_3\left(\frac{1}{8}\right)+b_4\left(\frac{1}{16}\right)+\cdots$$

So:

$$0.001_2=0\left(\frac{1}{2}\right)+0\left(\frac{1}{4}\right)+1\left(\frac{1}{8}\right)=0.125_{10}$$

12 Hand Algorithm for the Fractional Part

For a fractional number, compare it to:

$$\frac{1}{2},\frac{1}{4},\frac{1}{8},\frac{1}{16},\ldots$$

At each step:

- If the current number is greater than or equal to the comparison value, write `1` and subtract the comparison value.
- Otherwise, write `0`.

12.1 Example: 0.125

To six places:

Place	Comparison	Current Number	Output Bit
1	$1/2=0.5$	0.125	0
2	$1/4=0.25$	0.125	0
3	$1/8=0.125$	0.125	1
4	$1/16=0.0625$	0	0
5	$1/32=0.03125$	0	0
6	$1/64=0.015625$	0	0

Thus:

$$0.125_{10}=0.001000_2$$

12.2 Example: 0.95

For a less trivial example:

$$0.95_{10}$$

- $0.95 > 0.5$, so write 1 and subtract:

$$0.95 - 0.5 = 0.45$$

- $0.45 > 0.25$, so write 1 and subtract:

$$0.45 - 0.25 = 0.20$$

- $0.20 > 0.125$, so write 1 and subtract:

$$0.20 - 0.125 = 0.075$$

- $0.075 > 0.0625$, so write 1 and subtract:

$$0.075 - 0.0625 = 0.0125$$

So the binary fraction begins:

$$0.95_{10} \approx 0.1111\ldots_2$$

The expansion continues, and because many decimal fractions do not terminate in binary, this process often produces an approximation.

13 Python Function for the Fractional Part

```
def convert_fraction_to_binary(num, places):
    # Convert the fractional part of a number to binary.

    result = ""
    count = 0

    while num > 0 and count < places:
        comparison = 2 ** (-(count + 1))

        if num >= comparison:
            result = result + "1"
            num = num - comparison
        else:
            result = result + "0"

        count = count + 1

    while len(result) < places:
        result = result + "0"

    return result
```

14 Putting the Pieces Together

Now we combine the integer and fractional pieces.

```
def get_binary_rep(n, p):
    # Return a binary string representation of n using p fractional bits.

    front = int(n)
    back = n - front

    front_bin = convert_integer_to_binary(front)
    back_bin = convert_fraction_to_binary(back, p)

    result = front_bin + "." + back_bin

    return result
```

For:

```
get_binary_rep(3.125, 6)
```

the result is:

```
11.001000
```

15 From Binary Representation to Floating Point Form

Once we have a binary representation such as:

$$11.001000_2$$

we want to express it in normalized binary scientific notation:

$$1.xxxxx \times 2^E$$

For example:

$$11.001000_2 = 1.1001000_2 \times 2^1$$

Here the mantissa begins after the leading 1, the exponent is 1, and the leading 1 is implicit in normalized representation.

16 Step 5: Determine Mantissa and Exponent

Suppose the binary representation has two parts:

```
front_bin = "11"
back_bin = "001000"
```

Since the integer part is nonzero, this is a positive-exponent case.

16.1 Positive Exponent Case

If the part before the binary point is not zero, then the leading digit will always be 1.

For example:

$$10011010.11101_2$$

The decimal point must move left until only one digit remains before the point.

If:

```
front_bin = "10011010"
```

then:

$$\text{exponent} = \text{len}(\text{front\_bin}) - 1 = 8 - 1 = 7$$

The mantissa is built from:

$$\texttt{front\_bin[1:]} + \texttt{back\_bin}$$

because the first 1 is implicit and is not stored.

```
exponent = len(front_bin) - 1
mantissa = front_bin[1:] + back_bin
```

16.2 Negative Exponent Case

If the integer part is zero, then the number has the form:

$$0.00101101101_2$$

Here we must search the fractional part to find the first `1`.

For example:

$$0.00101101101_2 = 1.01101101_2 \times 2^{-3}$$

Thus the exponent is negative.

A clean Python approach is:

```
first_one = back_bin.find("1")
exponent = -(first_one + 1)
mantissa = back_bin[first_one + 1:]
```

If:

```
back_bin = "00101101101"
```

then `first_one = 2`, so:

$$E = -(2 + 1) = -3$$

17 Mantissa Padding and Truncation

The mantissa field must contain exactly 23 bits.

Therefore:

- if the mantissa is too short, pad it with zeroes;
- if the mantissa is too long, truncate it.

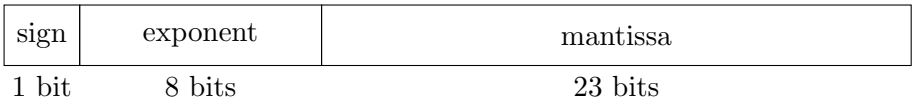
```
if len(mantissa) < 23:
    mantissa = mantissa + (23 - len(mantissa)) * "0"

if len(mantissa) > 23:
    mantissa = mantissa[0:23]
```

18 Step 6: Convert to 32-Bit Floating Point Representation

A 32-bit single precision style floating point representation consists of:

Field	Number of Bits	Meaning
Sign	1	positive or negative
Exponent	8	biased exponent
Mantissa	23	fractional significand bits



19 The Sign Bit

The sign bit is `0` for positive numbers and `1` for negative numbers.

```
if n < 0:
    sign_bit = "1"
    n = abs(n)
else:
    sign_bit = "0"
```

20 The Exponent Field

Single precision stores the exponent using a bias of 127.

If the actual exponent is E, then the stored exponent is:

$$E + 127$$

For example, if $E = 7$, then:

$$E_{\text{stored}} = 7 + 127 = 134$$

Then 134 is converted to an 8-bit binary number.

```
exp = int(exponent) + 127
exp_binary_rep = convert_integer_to_binary(exp)
```

If the exponent binary representation has fewer than 8 bits, we pad with leading zeroes:

```
if len(exp_binary_rep) < 8:
    exp_binary_rep = (8 - len(exp_binary_rep)) * "0" + exp_binary_rep
```

21 Final 32-Bit String

Finally, the 32-bit representation is assembled as:

$$\text{sign bit} + \text{exponent bits} + \text{mantissa bits}$$

```
float_rep = sign_bit + exp_binary_rep + mantissa
```

22 Complete Program

```
def convert_integer_to_binary(num):
    # Convert a nonnegative integer to a binary string.

    if num == 0:
        return "0"

    result = ""

    while num > 0:
        digit = num % 2
        result = str(digit) + result
        num = num // 2

    return result


def convert_fraction_to_binary(num, places):
    # Convert a fractional number between 0 and 1 to binary.

    result = ""
    count = 0

    while num > 0 and count < places:
        comparison = 2 ** (-(count + 1))

        if num >= comparison:
            result = result + "1"
            num = num - comparison
        else:
            result = result + "0"

        count = count + 1
```

```
    while len(result) < places:
        result = result + "0"

    return result


def get_binary_rep(n, p):
    # Return binary representation of n using p fractional places.

    front = int(n)
    back = n - front

    front_bin = convert_integer_to_binary(front)
    back_bin = convert_fraction_to_binary(back, p)

    return front_bin + "." + back_bin


def normalize_binary_rep(front_bin, back_bin):
    # Return exponent and mantissa bits from binary pieces.

    if front_bin != "0":
        exponent = len(front_bin) - 1
        mantissa = front_bin[1:] + back_bin
    else:
        first_one = back_bin.find("1")

        if first_one == -1:
            exponent = 0
            mantissa = "0" * 23
        else:
            exponent = -(first_one + 1)
            mantissa = back_bin[first_one + 1:]

    if len(mantissa) < 23:
        mantissa = mantissa + (23 - len(mantissa)) * "0"

    if len(mantissa) > 23:
        mantissa = mantissa[0:23]

    return exponent, mantissa


def get_32_bit_float_rep(n, p):
    # Construct a simplified 32-bit floating point representation.

    if n < 0:
        sign_bit = "1"
        n = abs(n)
    else:
        sign_bit = "0"

    front = int(n)
    back = n - front
```

```
    front_bin = convert_integer_to_binary(front)
    back_bin = convert_fraction_to_binary(back, p)

    exponent, mantissa = normalize_binary_rep(front_bin, back_bin)

    exp = exponent + 127
    exp_binary_rep = convert_integer_to_binary(exp)

    if len(exp_binary_rep) < 8:
        exp_binary_rep = (8 - len(exp_binary_rep)) * "0" + exp_binary_rep

    if len(exp_binary_rep) > 8:
        raise ValueError("Exponent does not fit in 8 bits.")

    return sign_bit + exp_binary_rep + mantissa


n = float(input("Enter your floating point value: "))
p = int(input("Enter the number of binary places in the result: "))

binary_rep = get_binary_rep(abs(n), p)
float_rep = get_32_bit_float_rep(n, p)

print("Binary representation:", binary_rep)
print("32-bit floating point representation:", float_rep)
print("Grouped:")
print(float_rep[0], float_rep[1:9], float_rep[9:])
```

23 Testing the Program

For:

$$n = 3.125$$

we know:

$$3.125_{10} = 11.001_2$$

Normalized:

$$11.001_2 = 1.1001_2 \times 2^1$$

So:

- sign bit is `0`;
- exponent is 1;
- biased exponent is $1 + 127 = 128$;
- exponent bits are `10000000`;
- mantissa begins `1001` and is padded with zeroes.

Thus the 32-bit representation should be:

$$0\ 10000000\ 10010000000000000000000$$

or:

```
01000000010010000000000000000000
```

24 Important Caveats

This program is educational. It deliberately avoids several complications of the full IEEE-754 standard, including:

- rounding instead of truncation,
- subnormal numbers,
- signed zero,
- infinity,
- NaN,
- overflow and underflow handling,
- exact binary behavior of Python's built-in floats.

Nevertheless, it captures the central idea:

$$\text{floating point number} = (-1)^s \times 1.\text{mantissa} \times 2^E$$

25 Key Takeaways

- A floating point number can be separated into integer and fractional parts.
- Integer parts can be converted to binary using repeated division by 2.
- Fractional parts can be converted using repeated comparison to powers of $1/2$.
- Binary floating point numbers are normalized into the form $1.xxxxx \times 2^E$.
- A 32-bit representation stores 1 sign bit, 8 exponent bits, and 23 mantissa bits.
- The exponent is stored with a bias of 127.
- Python string operations are useful for constructing binary representations manually.